Ad Soyad: _____	**Tarih:** _____
Bölüm 10 – Ünite 3 – *Thread Havuzu Tasarımı – Çalışma Kağıdı*	

Süre

Tahmini süre: **45 dakika**, Toplam: **50 puan**.

Bölüm 1: Tasarım Problemleri

(20 dakika – 30 puan)

1. **İşçi sayısı seçimi**: 16 mantıksal çekirdekli bir sunucuda çalışacak bir thread havuzu için aşağıdaki iş yükü türlerinde kaç işçi seçerdin ve neden? (10 puan)
   - CPU-yoğun matris çarpımı (her görev 100 ms CPU):
   - I/O-yoğun ağ istekleri (her görev 500 ms beklemek):
   - Karma iş (50% CPU, 50% I/O):

2. **Granülerlik ve batch**: 10.000 öğrencinin GPA'sı hesaplanacak; her GPA hesaplaması 0.3 ms sürüyor. Bu görevleri thread havuzuna göndermek için iki seçenek var: **(A)** her öğrenci için ayrı submit; **(B)** 100 öğrencilik batch'ler halinde 100 submit. Hangi yaklaşım daha hızlıdır ve neden? (10 puan)

3. **Deadlock analizi**: 2 işçili bir thread havuzunda şu kod yazıldı:

```
auto f1 = havuz.submit([&] {
    auto f2 = havuz.submit([] { return uzunIs(); });
    return f2.get();
});
```

   Bu kod 2 işçili havuzda deadlock yaratır mı? 4 işçili havuzda? Havuz boyutunu değiştirmeden nasıl düzeltirsiniz? (10 puan)

Bölüm 2: Performans Analizi

(15 dakika – 10 puan)

Aşağıdaki benchmark sonuçları 8 çekirdekli bir makinede ölçülmüştür:

İşçi Sayısı	Süre (ms)	Hızlanma
1	3200	1.0x
2	1700	1.88x
4	900	3.56x
8	520	6.15x
16	510	6.27x
32	580	5.52x

1. 8 işçide 6.15x hızlanma elde edildi; ideal olan 8x olurdu. Fark neden kaynaklanıyor? (Amdahl yasasına atıf yaparak açıklayın.) (5 puan)

2. 32 işçide süre neden **artıyor**? Bu fenomeni nasıl açıklarsınız? (5 puan)

Bölüm 3: Kod Tamamlama

(10 dakika – 10 puan)

1. Aşağıdaki `ThreadPool::submit` uygulamasındaki eksiklikleri tamamlayın:

```
template<typename F, typename... Args>
auto ThreadPool::submit(F&& f, Args&&... args)
    -> std::future<_______________>  // (a) donus tipi
{
    using R = std::invoke_result_t<F, Args...>;
    auto gorev = std::make_shared<______________>(  // (b) sarmalayici
        std::bind(std::forward<F>(f), std::forward<Args>(args)...));

    std::future<R> gelecek = gorev->________________;  // (c) future al

    kuyruk_.push([gorev]() { (*gorev)(); });

    return gelecek;
}
```

(a) _____

(b) _____

(c) _____